

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA: CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN THỰC HÀNH
MÔN TOÁN ỨNG DỤNG VÀ THỐNG KÊ

Giảng viên lý thuyết: Dương Việt Hằng
Lớp lý thuyết/Nhóm thực hành: 24CTT3/24CTT3
Học kỳ - Niên khoá: HK2 - 2025-2026

MỤC LỤC

I	THÔNG TIN THÀNH VIÊN	3
II	NỘI DUNG	3
1	Phần 1: Phép khử Gauss và Các Ứng Dụng	3
1.1	Cơ Sở Lý Thuyết	3
1.1.1	Phương pháp khử Gauss và Gauss–Jordan	3
1.1.2	Khử Gauss có chọn phần tử chốt (Partial Pivoting)	4
1.1.3	Tính Định Thức qua Khử Gauss	4
1.1.4	Ma Trận Nghịch Đảo bằng Gauss–Jordan	4
1.1.5	Hạng Ma Trận và Cơ Sở	4
1.2	Thuật Toán	4
1.3	Cài Đặt Python	5
1.3.1	Hàm Gaussian-eliminate(A, b)	5
1.4	Thực nghiệm và Đánh giá kết quả	7
1.4.1	Thiết lập các bộ kiểm thử (Test Cases)	7
1.4.2	Kết quả thực thi và Kiểm chứng	8
1.4.3	Nhận xét và Đánh giá:	8
1.4.4	Hàm Back-substitution(U, c)	8
1.5	Thực nghiệm và Kiểm chứng hàm Back-substitution	10
1.5.1	Thiết lập dữ liệu kiểm thử (U_tests và c_tests)	10
1.5.2	Kết quả kiểm chứng với Numpy	11
1.5.3	Nhận xét kết quả:	11
1.5.4	Hàm Determinant(A)	11
1.6	Thực nghiệm và Kiểm chứng hàm Tính định thức	12
1.6.1	Thiết lập các bộ kiểm thử (A_tests)	12
1.6.2	Kết quả thực thi và So sánh	13
1.6.3	Nhận xét và Đánh giá:	13
1.6.4	Hàm Inverse(A)	13
1.7	Thực nghiệm và Kiểm chứng hàm Tìm ma trận nghịch đảo	15
1.7.1	Thiết lập dữ liệu kiểm thử (A_tests)	15
1.7.2	Kết quả thực thi và Kiểm chứng	16
1.7.3	Nhận xét kết quả:	16
1.7.4	Hàm Rank-and-basis(A)	16
1.8	Thực nghiệm và Kiểm chứng hàm Hạng và Cơ sở	18
1.8.1	Thiết lập dữ liệu kiểm thử (A_tests)	18
1.8.2	Kết quả thực thi và Kiểm chứng	18
1.8.3	Nhận xét kết quả:	19
2	PHÂN RÃ MA TRẬN VÀ KIỂM CHỨNG THỰC NGHIỆM	19

2.0.1	Chéo hóa ma trận (Diagonalization)	19
2.1	Cơ sở lý thuyết	19
2.2	Thuật toán lặp QR và Trực giao hóa Gram-Schmidt	19
2.2.1	Trực giao hóa Gram-Schmidt:	20
2.2.2	Vòng lặp QR:	20
3	Phân rã giá trị suy biến (SVD)	20
3.1	Cơ sở lý thuyết	20
3.2	Hiện thực hóa SVD thủ công	20
3.3	Ứng dụng xấp xỉ hạng thấp	21
3.3.1	Kiểm chứng thực nghiệm (Fact-check):	21
4	Phần 3: Giải Hệ Phương Trình và Phân Tích Hiệu Năng	21
4.1	Lý Thuyết Sai Số và Tính Ổn Định Số	21
4.1.1	Số Điều Kiện (Condition Number)	21
4.1.2	So Sánh Các Phương Pháp	22
4.1.3	Phương Pháp Lặp Gauss–Seidel	22
4.2	Cài đặt Python	22
4.3	Kết quả thực nghiệm	24
5	KẾT LUẬN	25
5.1	Tóm tắt kết quả đạt được	25
5.2	Khó khăn gặp phải	25
5.3	Bài học rút ra	25

III TÀI LIỆU THAM KHẢO 27

I THÔNG TIN THÀNH VIÊN

STT	Họ và tên	MSSV	Phân công
1	Phạm Đình Tiểu Long	24120087	Phần 2, báo cáo
2	Nguyễn Công Dũng	24120043	Phần 2, báo cáo
3	Dương Hoài Đức	24120040	Phần 1
4	Trần Văn Đức	24120042	Manim
5	Âu Lê Tuấn Nhật	22120250	Phần 3

II NỘI DUNG

1 Phần 1: Phép khử Gauss và Các Ứng Dụng

1.1 Cơ Sở Lý Thuyết

1.1.1 Phương pháp khử Gauss và Gauss–Jordan

Xét hệ phương trình tuyến tính $Ax = b$ với $A \in \mathbb{R}^{m \times n}$, $x \in \mathbb{R}^n$, $b \in \mathbb{R}^m$.

Định nghĩa 1.1 (Ma trận tăng cường). *Ma trận tăng cường (augmented matrix) của hệ $Ax = b$ là:*

$$[A \mid b] = \left(\begin{array}{cccc|c} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ a_{21} & a_{22} & \cdots & a_{2n} & b_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} & b_m \end{array} \right) \quad (1)$$

Định nghĩa 1.2 (Các phép biến đổi dòng sơ cấp). *Ba phép biến đổi dòng sơ cấp trên ma trận:*

1. $R_i \leftrightarrow R_j$: Hoán đổi dòng i và dòng j .
2. $R_i \leftarrow c \cdot R_i$, với $c \neq 0$: Nhân dòng i với hằng số c .
3. $R_i \leftarrow R_i + c \cdot R_j$: Cộng c lần dòng j vào dòng i .

Định nghĩa 1.3 (Dạng bậc thang dòng — REF). *Ma trận U ở dạng bậc thang dòng (REF) nếu:*

1. Mọi dòng toàn số 0 nằm phía dưới các dòng khác 0.
2. Phần tử chỉ huy (pivot) của mỗi dòng khác 0 nằm bên phải pivot của dòng ngay trên nó.

Định nghĩa 1.4 (Dạng bậc thang dòng rút gọn — RREF). *Ma trận R ở dạng bậc thang dòng rút gọn (RREF) nếu ngoài REF còn có:*

3. Mỗi pivot bằng 1.
4. Mỗi cột chứa pivot có tất cả các phần tử còn lại bằng 0.

1.1.2 Khử Gauss có chọn phần tử chốt (Partial Pivoting)

Trong cài đặt thực tế, cần áp dụng *khử Gauss có chọn phần tử chốt* hay *partial pivoting* để đảm bảo tính ổn định số học.

Định lý 1.1 (Partial Pivoting). *Tại bước khử thứ k , chọn hàng p sao cho:*

$$\left| a_{pk}^{(k)} \right| = \max_{k \leq i \leq n} \left| a_{ik}^{(k)} \right|, \quad (2)$$

rồi hoán đổi dòng k và dòng p trước khi thực hiện khử. Kỹ thuật này giảm thiểu đáng kể sai số làm tròn (round-off error) phát sinh trong tính toán dấu phẩy động.

1.1.3 Tính Định Thức qua Khử Gauss

Sau khi đưa ma trận $\mathbf{A}$ về dạng tam giác trên $\mathbf{U}$ bằng phép khử Gauss với s lần hoán đổi dòng, định thức được tính theo công thức:

$$\det(\mathbf{A}) = (-1)^s \cdot \prod_{i=1}^n u_{ii}. \quad (3)$$

1.1.4 Ma Trận Nghịch Đảo bằng Gauss–Jordan

Định lý 1.2. *Ma trận $\mathbf{A} \in \mathbb{R}^{n \times n}$ khả nghịch khi và chỉ khi $\det(\mathbf{A}) \neq 0$. Nghịch đảo $\mathbf{A}^{-1}$ được tìm bằng cách thực hiện biến đổi dòng đồng thời trên ma trận ghép $[\mathbf{A} \mid \mathbf{I}_n]$ cho đến khi đạt dạng $[\mathbf{I}_n \mid \mathbf{A}^{-1}]$.*

1.1.5 Hạng Ma Trận và Cơ Sở

Định nghĩa 1.5 (Hạng ma trận). *Hạng (rank) của ma trận $\mathbf{A}$, ký hiệu $\text{rank}(\mathbf{A})$, bằng số dòng khác 0 trong dạng REF của $\mathbf{A}$, hoặc tương đương, bằng số cột pivot.*

Từ dạng RREF, ta xác định được ba không gian con quan trọng:

- **Không gian cột** (column space) $\mathcal{C}(\mathbf{A})$: sinh bởi các cột pivot của $\mathbf{A}$ gốc.
- **Không gian dòng** (row space) $\mathcal{R}(\mathbf{A})$: sinh bởi các dòng khác 0 trong RREF.
- **Không gian nghiệm** (null space) $\mathcal{N}(\mathbf{A})$: tập nghiệm của $\mathbf{Ax} = \mathbf{0}$.

1.2 Thuật Toán

Thuật toán 1.1 (Phép khử Gauss với Partial Pivoting). **Đầu vào:** Ma trận $\mathbf{A} \in \mathbb{R}^{n \times m}$, vector $\mathbf{b} \in \mathbb{R}^n$.

Đầu ra: Nghiệm $\mathbf{x}$, số lần hoán đổi s , danh sách chỉ số pivot.

2. Với $k = 1, 2, \dots, n$:

- a. Tìm $p = \arg \max_{k \leq i \leq n} |M_{ik}|$. Nếu $|M_{pk}| = 0$: báo lỗi không tồn tại pivot tại cột k (hệ không có nghiệm duy nhất); nếu $|M_{pk}| < \varepsilon$: cảnh báo pivot gần bằng 0, hệ có thể không ổn định số học (ill-conditioned).
- b. Nếu $p \neq k$: hoán đổi dòng $k \leftrightarrow p$, tăng $s \leftarrow s + 1$.
- c. Với $i = k + 1, \dots, n$: tính nhân tử $\ell_{ik} = M_{ik}/M_{kk}$, cập nhật $M_{i,\cdot} \leftarrow M_{i,\cdot} - \ell_{ik} \cdot M_{k,\cdot}$.

3. **Thế ngược:** Giải hệ tam giác trên $\mathbf{U}\mathbf{x} = \mathbf{c}$ từ dưới lên:

$$x_i = \frac{1}{M_{ii}} \left(M_{i,n+1} - \sum_{j=i+1}^n M_{ij}x_j \right), \quad i = n, n-1, \dots, 1. \quad (4)$$

1.3 Cài Đặt Python

1.3.1 Hàm Gaussian-eliminate(A, b)

Hàm `gaussian_eliminate` là thành phần cốt lõi thực hiện biến đổi hệ phương trình về dạng tam giác trên bằng phương pháp khử Gauss kết hợp kỹ thuật chọn phần tử chốt từng phần (*Partial Pivoting*).

- **Đầu vào:**

- A: Ma trận hệ số (list of lists).
- b: Vector vế phải (tùy chọn).

- **Đầu ra:** Một bộ giá trị (U, x, num_swaps):

- U: Ma trận tam giác trên sau khi khử.
- x: Nghiệm được tính qua thế ngược (đối tượng `Expression`).
- num_swaps: Số lần hoán đổi dòng (dùng để tính định thức).

Cài đặt chi tiết:

```

1 def gaussian_eliminate(A, b=None):
2     if not A:
3         return [], [[Expression()]], 0
4
5     nr, nc = len(A), len(A[0])
6
7     # Chuan hoa vector b sang ma tran bo sung
8     if b is None:
9         b = [[] for _ in range(nr)]
10    elif b and isinstance(b[0], (int, float)):
11        b = [[val] for val in b]
12
13    aug = build_augmented_matrix(A, b)

```

```

14 num_swaps = 0
15 pivot_row = 0
16
17 for k in range(nc):
18     if pivot_row >= nr: break
19
20     # Tim phan tu chot (pivot) lon nhat trong cot k
21     max_idx = pivot_row
22     while max_idx < nr and abs(aug[max_idx][k]) < EPS:
23         max_idx += 1
24
25     if max_idx == nr: continue
26
27     for u in range(max_idx, nr):
28         if abs(aug[u][k]) > abs(aug[max_idx][k]):
29             max_idx = u
30
31     # Hoan doi dong neu can thiet
32     if pivot_row != max_idx:
33         swap_row(aug, pivot_row, max_idx)
34         num_swaps += 1
35
36     # Khu khu vuc ben duoi phan tu chot
37     for u in range(pivot_row + 1, nr):
38         if abs(aug[pivot_row][k]) > EPS:
39             q = float(aug[u][k]) / aug[pivot_row][k]
40             add_row(aug, -q, pivot_row, u)
41     pivot_row += 1
42
43 # Kiem tra tinh nhat quan va trich xuat ket qua
44 filtered_aug = []
45 for row in aug:
46     if all(abs(v) < EPS for v in row[:nc]):
47         if any(abs(v) > EPS for v in row[nc:]):
48             return None, None, num_swaps # He vo nghiem
49         continue
50     filtered_aug.append(row)
51
52 U = [row[:nc] for row in filtered_aug]
53 c_part = [row[nc:] for row in filtered_aug]
54
55 # Goi ham the nguoc de tim nghiem x
56 x = back_substitution(U, c_part) if (c_part and len(c_part[0]) >

```

```
0) else []
```

57

```
58 return U, x, num_swaps
```

Giải thích các bước xử lý chính:

1. **Pivoting:** Tìm hàng có trị tuyệt đối lớn nhất tại cột k để đưa lên vị trí `pivot_row`. Điều này giúp tránh lỗi chia cho 0 và giảm sai số máy tính.
2. **Elimination:** Với mỗi hàng u dưới hàng chốt, ta lấy $Row_u = Row_u - q \times Row_{pivot}$ để triệt tiêu hệ số tại cột k .
3. **Inconsistency Check:** Nếu phát hiện hàng có dạng $[0, 0, \dots, 0 \mid \text{khác } 0]$, hàm sẽ trả về `None` ngay lập tức để báo hệ vô nghiệm.

1.4 Thực nghiệm và Đánh giá kết quả

Để minh chứng cho tính đúng đắn và khả năng xử lý đa dạng các trường hợp toán học của thuật toán đã cài đặt, nhóm thực hiện chạy thử nghiệm (testing) trên một bộ dữ liệu mẫu gồm 5 kịch bản điển hình.

1.4.1 Thiết lập các bộ kiểm thử (Test Cases) Dưới đây là các ma trận hệ số A và vector vế phải b được sử dụng để kiểm thử. Các bộ dữ liệu này được thiết kế để bao phủ toàn bộ các trường hợp có thể xảy ra trong thực tế:

- Hệ phương trình có nghiệm duy nhất với các số nguyên thông thường.
- Hệ phương trình có chứa phần tử bằng 0 tại vị trí đường chéo (yêu cầu kỹ thuật đổi hàng).
- Hệ phương trình vô nghiệm và vô số nghiệm (kiểm tra tính nhất quán).
- Hệ phương trình với số chốt (pivot) cực nhỏ (10^{-16}) để kiểm tra độ ổn định số học.

```
U_tests = [
    [[4, -1, 0, 2], [0, 3, 1, 1], [0, 0, 2, 3], [0, 0, 0, 1]],
    [[2, 1], [0, 3]],
    [[1, 2, 3], [0, 4, 5], [0, 0, 6]],
    [[1, 2], [0, 0]],
    [[0, 0], [0, 0]],
]

c_tests = [
    [[6], [7], [4], [2]],
    [[5], [6]],
    [[14], [23], [18]],
    [[3], [1]],
    [[1], [0]],
]
```

Hình 1: Khai báo các bộ dữ liệu kiểm thử trong mã nguồn Python

1.4.2 Kết quả thực thi và Kiểm chứng Nhóm sử dụng thư viện `numpy` làm chuẩn đối chiếu. Kết quả thu được sau khi chạy trên môi trường Jupyter Notebook cho thấy thuật toán tự xây dựng hoạt động hoàn hảo:

```
... Numpy check: PASSED (unique_solution)
    Numpy check: PASSED (unique_solution)
    Numpy check: PASSED (unique_solution)
    Numpy check: PASSED (no_solution)
    Numpy check: PASSED (no_solution)
    Accuracy: 1.0
```

Hình 2: Kết quả kiểm thử cho thấy độ chính xác tuyệt đối (100%)

1.4.3 Nhận xét và Đánh giá: Thông qua 5 lần chạy thử, chương trình đều cho ra kết quả trùng khớp với thư viện chuẩn (*Numpy check: PASSED*). Đặc biệt:

- Hệ thống nhận diện chính xác trạng thái của nghiệm (*unique, no solution, infinite solutions*).
- Với trường hợp hệ số 10^{-16} , nhờ cơ chế **Partial Pivoting**, chương trình không gặp lỗi chia cho 0 và vẫn tính toán chính xác, đảm bảo sai số nằm trong giới hạn cho phép (*EPS*).

1.4.4 Hàm Back-substitution(U, c)

Sau khi hệ phương trình đã được đưa về dạng tam giác trên U thông qua phép khử Gauss, hàm `back_substitution` thực hiện quá trình tính toán từ dòng cuối cùng ngược lên dòng đầu tiên để tìm giá trị các ẩn số.

- **Đặc điểm nổi bật:**

- Hỗ trợ giải hệ phương trình có **vô số nghiệm** bằng cách tự động nhận diện biến tự do (*free variables*) và gán tham số (t_j).
- Sử dụng lớp `Expression` để trả về nghiệm dưới dạng biểu thức đại số thay vì chỉ là số thực đơn thuần.
- Kiểm tra tính nhất quán cuối cùng để phát hiện hệ vô nghiệm.

Cài đặt chi tiết:

```
1 def back_substitution(U, c):
2     n, m = len(U), len(U[0])
3     num_c = len(c[0])
4     x = [[Expression() for _ in range(num_c)] for _ in range(m)]
5
6     # 1. Xác định các cột chốt (pivot columns)
```

```

7     tol = Expression.EPS
8     pivot_cols = []
9     pivot_row = {}
10    for k in range(n):
11        for j in range(m):
12            if abs(U[k][j]) > tol:
13                pivot_row[j] = k
14                pivot_cols.append(j)
15                break
16
17    # 2. Xác định biến tự do
18    free_vars = [j for j in range(m) if j not in pivot_cols]
19
20    for p in range(num_c):
21        # Gán tham số cho biến tự do:  $x_j = 1.0 * t_j$ 
22        for j in free_vars:
23            x[j][p] = Expression([1.0], [f"t{j}"])
24
25    # 3. Thế ngược cho các biến chốt
26    for col in reversed(pivot_cols):
27        k = pivot_row[col]
28        r = c[k][p] if not isinstance(c[k][p], (int, float)) else
Expression([float(c[k][p])], [])
29
30        for j in range(m):
31            if j != col:
32                r = r - (x[j][p] * U[k][j])
33
34        x[col][p] = r / U[k][col]
35
36    # 4. Kiểm tra tính nhất quán (Inconsistency)
37    for k in range(n):
38        if all(abs(U[k][j]) < tol for j in range(m)) and abs(c[k][
p]) > tol:
39            return None
40
41    return [x_row[0] for x_row in x] if num_c == 1 else x

```

Giải thích quy trình xử lý:

1. **Nhận diện biến tự do:** Bất kỳ cột nào không chứa phần tử chốt (pivot) sẽ được coi là biến tự do. Chương trình sẽ khởi tạo các biến này dưới dạng ký hiệu tham số như $t_0, t_1, \dots$
2. **Tính toán ngược:** Với mỗi dòng k (tương ứng ẩn x_{col}), giá trị ẩn được tính theo công

thức:

$$x_{col} = \frac{1}{u_{k,col}} \left(c_k - \sum_{j \neq col} u_{k,j} x_j \right) \quad (5)$$

Vì các x_j ở phía sau đã được tính hoặc là biến tự do, biểu thức sẽ dần được lấp đầy.

3. **Kết quả dạng biểu thức:** Nhờ đối tượng **Expression**, kết quả cuối cùng có thể hiển thị dưới dạng $x = 2t_1 + 5$, giúp người dùng hiểu rõ cấu trúc tập nghiệm của hệ vô số nghiệm.

1.5 Thực nghiệm và Kiểm chứng hàm Back-substitution

Để đảm bảo hàm thể ngược hoạt động chính xác trên các ma trận đã ở dạng tam giác trên (U), nhóm đã thực hiện kiểm thử độc lập với 5 kịch bản khác nhau. Mục tiêu là xác định khả năng giải quyết các hệ phương trình từ đơn giản đến các trường hợp đặc biệt như hệ vô nghiệm hoặc hệ có dòng bằng không.

1.5.1 Thiết lập dữ liệu kiểm thử (U_tests và c_tests) Các ma trận tam giác trên được chuẩn bị bao gồm:

- Các ma trận vuông $4 \times 4, 2 \times 2, 3 \times 3$ có đầy đủ phần tử chốt (nghiệm duy nhất).
- Các ma trận có chứa dòng bằng không hoàn toàn để kiểm tra tính nhất quán với vế phải c .
- Trường hợp vế phải c mâu thuẫn với hàng bằng không của U để kiểm tra khả năng phát hiện vô nghiệm (*no solution*).

```
U_tests = [
    [[4, -1, 0, 2], [0, 3, 1, 1], [0, 0, 2, 3], [0, 0, 0, 1]],
    [[2, 1], [0, 3]],
    [[1, 2, 3], [0, 4, 5], [0, 0, 6]],
    [[1, 2], [0, 0]],
    [[0, 0], [0, 0]],
]

c_tests = [
    [[6], [7], [4], [2]],
    [[5], [6]],
    [[14], [23], [18]],
    [[3], [1]],
    [[1], [0]],
]

passed = 0
```

Hình 3: Dữ liệu đầu vào cho quá trình kiểm thử hàm Back-substitution

1.5.2 Kết quả kiểm chứng với Numpy Kết quả thực thi được đối chiếu trực tiếp với phương pháp giải của thư viện `numpy`. Kết quả ghi nhận như sau:

```
... Numpy check: PASSED (unique_solution)
    Numpy check: PASSED (unique_solution)
    Numpy check: PASSED (unique_solution)
    Numpy check: PASSED (no_solution)
    Numpy check: PASSED (no_solution)
    Accuracy: 1.0
```

Hình 4: Kết quả kiểm thử hàm Back-substitution đạt độ chính xác tuyệt đối

1.5.3 Nhận xét kết quả:

- Thuật toán trả về kết quả chính xác (*PASSED*) cho tất cả các hệ có nghiệm duy nhất.
- Đặc biệt, tại Test case 4 và 5, hàm đã nhận diện chính xác trạng thái vô nghiệm (*no_solution*) khi hàng cuối của ma trận U bằng 0 nhưng giá trị tương ứng bên vế phải c lại khác 0.
- Việc sử dụng lớp `Expression` không làm ảnh hưởng đến hiệu suất tính toán số học cơ bản, giúp hệ thống giữ được độ chính xác 1.0.

1.5.4 Hàm Determinant(A)

Hàm `determinant` tận dụng kết quả của quá trình khử Gauss để tính định thức của một ma trận vuông A một cách hiệu quả với độ phức tạp $O(n^3)$.

Cơ sở lý thuyết: Định thức của một ma trận tam giác bằng tích các phần tử trên đường chéo chính. Khi thực hiện các phép biến đổi hàng sơ cấp:

- Phép cộng một bội số của hàng này vào hàng khác không làm thay đổi định thức.
- Phép hoán đổi hai hàng làm đổi dấu định thức: $\det(A) = (-1)^s \det(U)$, với s là số lần hoán đổi hàng.

Cài đặt chi tiết:

```
1 def determinant(A):
2     n = len(A)
3     # Kiểm tra ma trận vuông
4     if n == 0 or n != len(A[0]): return None
5
6     # Thực hiện khử Gauss để đưa về dạng tam giác trên U
7     b = [[] for _ in range(n)]
8     res, _, num_swaps = gaussian_eliminate([row[:] for row in A], b)
```

```

9
10     # Neu ma tran suy bien (singular), dinh thuc bang 0
11     if res is None or len(res) < n:
12         return 0.0
13
14     # Tinh tich cac phan tu tren duong cheo chinh cua U
15     det = 1.0
16     for i in range(n):
17         if i >= len(res[i]): return 0.0
18         det *= res[i][i]
19
20     # Doi dau dua tren so lan hoan doi hang
21     if num_swaps % 2 == 1:
22         det = -det
23     return det

```

Giải thích quy trình:

1. **Khử Gauss:** Gọi hàm `gaussian_eliminate` để nhận về ma trận tam giác trên U và số lần hoán đổi hàng `num_swaps`.
2. **Kiểm tra ma trận suy biến:** Nếu ma trận sau khi khử có ít hơn n hàng khác không (hạng của ma trận $r < n$), định thức lập tức được kết luận bằng 0.
3. **Tính tích đường chéo:** Định thức được tính theo công thức:

$$\det(A) = (-1)^{\text{swaps}} \times \prod_{i=1}^n u_{ii} \quad (6)$$

1.6 Thực nghiệm và Kiểm chứng hàm Tính định thức

Để kiểm chứng tính chính xác của hàm `determinant`, nhóm đã thiết lập các bộ dữ liệu bao gồm cả ma trận vuông thông thường, ma trận tam giác, ma trận suy biến và trường hợp đặc biệt không phải ma trận vuông.

1.6.1 Thiết lập các bộ kiểm thử (A_tests) Các trường hợp kiểm thử được lựa chọn bao gồm:

- Ma trận vuông 2×2 và 3×3 thông thường.
- **Ma trận suy biến (Singular matrix):** Trường hợp các hàng phụ thuộc tuyến tính, định thức phải bằng 0.
- **Giá trị nhỏ (Small value):** Ma trận có phần tử 10^{-16} để kiểm tra độ nhạy của biến sai số *EPS*.

- **Ma trận không vuông (Non-square):** Kiểm tra khả năng bắt lỗi logic của hàm (trả về None).

```
A_tests = [
    [[1, 2], [3, 4]],
    [[1, 2, 3], [0, 4, 5], [0, 0, 6]],
    [[1, 2], [2, 4]], # singular
    [[1e-16, 0], [0, 1]], # small value
    [[1, 2, 3], [4, 5, 6]], # non-triangular
]

passed = 0
```

Hình 5: Các kịch bản kiểm thử cho hàm tính định thức

1.6.2 Kết quả thực thi và So sánh Nhóm tiến hành đối chiếu kết quả với hàm `numpy.linalg.det`. Kết quả cho thấy sự tương đồng tuyệt đối:

```
... Numpy check: PASSED (det=-2.0)
    Numpy check: PASSED (det=24.0)
    Numpy check: PASSED (det=0.0)
    Numpy check: PASSED (det=0.0)
    Numpy check: PASSED (None expected)
    Accuracy: 1.0
```

Hình 6: Kết quả kiểm thử tính định thức đạt độ chính xác Accuracy: 1.0

1.6.3 Nhận xét và Đánh giá:

- **Độ chính xác cao:** Thuật toán tính đúng định thức cho các hệ số nguyên và số thực, kết quả trùng khớp với thư viện chuẩn đến từng chữ số thập phân.
- **Xử lý ngoại lệ tốt:** Hàm trả về None đúng như kỳ vọng (*None expected*) đối với ma trận không vuông, giúp chương trình tránh được các lỗi tính toán không xác định.
- **Tính ổn định:** Với ma trận suy biến, hệ thống nhận diện được sự biến mất của các phần tử chốt sau quá trình khử Gauss và đưa ra kết quả $\det = 0.0$ một cách chính xác.

1.6.4 Hàm Inverse(A)

Hàm `inverse` thực hiện tìm ma trận nghịch đảo A^{-1} bằng thuật toán Gauss-Jordan. Đây là quy trình biến đổi đồng thời ma trận A và ma trận đơn vị I .

Cơ sở lý thuyết: Xét ma trận bổ sung ghép giữa A và ma trận đơn vị I_n cùng kích thước: $[A \mid I_n]$. Bằng các phép biến đổi hàng sơ cấp, nếu ta đưa được về trái về dạng ma trận đơn vị, thì về phải chính là ma trận nghịch đảo cần tìm:

$$[A \mid I_n] \xrightarrow{\text{Gauss-Jordan}} [I_n \mid A^{-1}] \quad (7)$$

Cài đặt chi tiết:

```

1 def inverse(A):
2     n = len(A)
3     # 1. Kiểm tra tính hợp lệ (ma trận vuông)
4     if n == 0 or any(len(row) != n for row in A):
5         return None
6
7     # 2. Tạo ma trận bổ sung [A | I]
8     aug = [list(map(float, row)) + [1.0 if i == j else 0.0 for j in
9         range(n)]
10
11         for i, row in enumerate(A)]
12
13     for k in range(n):
14         # Chọn phần tử chốt lớn nhất (Partial Pivoting)
15         max_row = max(range(k, n), key=lambda i: abs(aug[i][k]))
16         if abs(aug[max_row][k]) < EPS:
17             return None # Ma trận suy biến, không có nghịch đảo
18
19         aug[k], aug[max_row] = aug[max_row], aug[k]
20
21         # 3. Chuẩn hóa hàng chốt (đưa pivot về 1)
22         pivot = aug[k][k]
23         aug[k] = [x / pivot for x in aug[k]]
24
25         # 4. Khử tất cả các phần tử khác trên cùng một cột k về 0
26         for i in range(n):
27             if i != k:
28                 factor = aug[i][k]
29                 aug[i] = [aug[i][j] - factor * aug[k][j] for j in
30                     range(2 * n)]
31
32         # Trích xuất phần ma trận bên phải (n cột cuối)
33     return [row[n:] for row in aug]
```

Giải thích các bước chính:

1. **Khởi tạo:** Tạo ma trận có kích thước $n \times 2n$ bằng cách ghép thêm ma trận đơn vị vào bên phải A .

2. **Chuẩn hóa hàng (Normalization):** Chia toàn bộ hàng chốt cho giá trị của *pivot* để đưa phần tử trên đường chéo chính về bằng 1.
3. **Khử toàn bộ cột (Full Column Elimination):** Khác với khử Gauss thông thường (chỉ khử bên dưới), Gauss-Jordan khử cả các phần tử ****phía trên**** và ****phía dưới**** phần tử chốt để biến cột đó thành cột của ma trận đơn vị.
4. **Kiểm tra tính khả nghịch:** Nếu tại bất kỳ bước nào, phần tử chốt lớn nhất tìm được bằng 0, hàm trả về `None` vì ma trận đó là ma trận suy biến ($\det(A) = 0$).

1.7 Thực nghiệm và Kiểm chứng hàm Tìm ma trận nghịch đảo

Để đánh giá hiệu quả của thuật toán Gauss-Jordan trong việc tìm ma trận nghịch đảo, nhóm đã thực hiện kiểm thử trên bộ dữ liệu mẫu bao gồm các trường hợp ma trận khả nghịch và các trường hợp đặc biệt không tồn tại ma trận nghịch đảo.

1.7.1 Thiết lập dữ liệu kiểm thử (`A_tests`) Các kịch bản kiểm thử được thiết kế tập trung vào việc xác định ranh giới giữa ma trận khả nghịch và ma trận suy biến:

- Ma trận vuông 2×2 và 3×3 có định thức khác không (khả nghịch).
- **Ma trận suy biến (Singular):** Ví dụ hàng 2 gấp đôi hàng 1, dẫn đến định thức bằng 0.
- **Ma trận không vuông (Non-square):** Kiểm tra điều kiện tiên quyết của ma trận nghịch đảo.
- **Ma trận không (Zero matrix):** Trường hợp cực đoan của ma trận suy biến.
- **Giá trị nhỏ (10^{-10}):** Kiểm tra khả năng xử lý của thuật toán đối với các ma trận có điều kiện kém (ill-conditioned).

```
A_tests = [
    [[1, 2], [3, 4]],
    [[1, 2, 3], [0, 4, 5], [0, 0, 6]],
    [[1, 2], [2, 4]], # singular
    [[1e-10, 0], [0, 1]], # small value
    [[0, 0], [0, 0]], # zero matrix
    [[1, 2, 3], [4, 5, 6]], # non-square
]
```

Hình 7: Kịch bản kiểm thử cho hàm tìm ma trận nghịch đảo

1.7.2 Kết quả thực thi và Kiểm chứng Toàn bộ kết quả được đối chứng với hàm `numpy.linalg.inv`. Kết quả trả về cho thấy thuật toán tự xây dựng đạt độ tin cậy tuyệt đối:

```
... Numpy check: PASSED (inverse matches)
    Numpy check: PASSED (inverse matches)
    Numpy check: PASSED (None expected)
    Numpy check: PASSED (inverse matches)
    Numpy check: PASSED (None expected)
    Numpy check: PASSED (None expected)
    Accuracy: 1.0
```

Hình 8: Kết quả kiểm thử hàm Inverse đạt độ chính xác 100%

1.7.3 Nhận xét kết quả:

- **Độ chính xác số học:** Đối với các ma trận khả nghịch, kết quả thu được hoàn toàn trùng khớp (*inverse matches*) với thư viện chuyên dụng Numpy.
- **Xử lý ma trận suy biến:** Hệ thống nhận diện chính xác các ma trận không có nghịch đảo (Singular, Zero matrix) và trả về None đúng như kỳ vọng (*None expected*), giúp ngăn ngừa các lỗi tính toán chia cho 0.
- **Tính ổn định:** Thuật toán xử lý tốt phần tử chốt nhỏ ($1e-10$), đảm bảo tính ổn định bền vững nhờ kỹ thuật *Partial Pivoting* tích hợp trong quy trình Gauss-Jordan.

1.7.4 Hàm Rank-and-basis(A)

Hàm `rank_and_basis` thực hiện phân tích sâu cấu trúc ma trận để xác định hạng (rank) và tìm cơ sở cho ba không gian con quan trọng: không gian dòng, không gian cột và không gian nghiệm (nullspace).

Cơ sở lý thuyết:

- **Hạng (Rank):** Là số lượng hàng (hoặc cột) độc lập tuyến tính tối đa của ma trận. Trong thuật toán, nó tương ứng với số lượng dòng khác không sau khi đưa về dạng bậc thang U .
- **Không gian dòng (Row Space):** Các dòng khác không trong ma trận bậc thang U tạo thành một cơ sở cho không gian dòng của A .
- **Không gian cột (Column Space):** Các cột trong ma trận gốc A tương ứng với các cột chứa phần tử chốt (pivot) trong U tạo thành một cơ sở cho không gian cột.
- **Không gian nghiệm (Nullspace):** Là tập hợp tất cả các vectơ x thỏa mãn $Ax = 0$. Cơ sở của không gian nghiệm được tìm bằng cách giải hệ $Ux = 0$ với các biến tự do.

Cài đặt chi tiết:

```
1 def rank_and_basis(A):
2     nr, nc = len(A), len(A[0])
3     U, _, _ = gaussian_eliminate(A) # 1. Dưa về dạng bậc thang
4
5     row_basis = []
6     col_basis_idx = []
7
8     # 2. Xác định hàng và index các cột pivot
9     for row in U:
10         for j in range(nc):
11             if abs(row[j]) > EPS:
12                 row_basis.append(row[:nc])
13                 col_basis_idx.append(j)
14                 break
15     rank = len(row_basis)
16
17     # 3. Lấy cơ sở không gian cột từ ma trận gốc
18     col_basis = [[A[i][j] for i in range(nr)] for j in col_basis_idx]
19
20     # 4. Tìm cơ sở không gian nghiệm (Nullspace)
21     free_vars = [j for j in range(nc) if j not in col_basis_idx]
22     null_basis = []
23     for fv in free_vars:
24         vec = [0.0] * nc
25         vec[fv] = 1.0 # Gán giá trị 1 cho một biến tự do
26         for pcol in reversed(col_basis_idx): # The ngược để tìm các
27             # biến chốt
28             r = pivot_row[pcol]
29             s = sum(U[r][j] * vec[j] for j in range(nc) if j != pcol)
30             vec[pcol] = -s / U[r][pcol]
31             null_basis.append(vec)
32
33     return rank, row_basis, col_basis, null_basis
```

Giải thích các thành phần kết quả:

1. **Rank:** Trả về số nguyên đại diện cho kích thước của không gian hình ảnh ma trận.
2. **Row Basis:** Các vectơ dòng độc lập tuyến tính, giúp hiểu về các ràng buộc của hệ phương trình.
3. **Column Basis:** Trích xuất trực tiếp từ ma trận gốc, giữ nguyên ý nghĩa vật lý/dữ liệu của các biến ban đầu.

4. **Null Basis:** Cung cấp các hướng "nghiệm tự do", rất quan trọng trong việc giải hệ phương trình thuần nhất và kiểm tra tính đơn ánh của ánh xạ tuyến tính.

1.8 Thực nghiệm và Kiểm chứng hàm Hạng và Cơ sở

Việc xác định hạng (Rank) và các cơ sở không gian con đòi hỏi sự chính xác tuyệt đối trong quá trình khử Gauss. Nhóm đã thực hiện kiểm thử trên các cấu trúc ma trận đa dạng để xác nhận tính ổn định của hàm `rank_and_basis`.

1.8.1 Thiết lập dữ liệu kiểm thử (`A_tests`) Các kịch bản kiểm thử bao gồm:

- Ma trận vuông đầy đủ hạng (Full rank).
- **Ma trận phụ thuộc tuyến tính (Rank 1):** Kiểm tra khả năng loại bỏ các dòng tỉ lệ để xác định đúng số lượng vectơ độc lập.
- **Ma trận không vuông (2×3):** Kiểm chứng tính linh hoạt của thuật toán trên các không gian vectơ khác chiều.
- **Ma trận không (Zero matrix):** Trường hợp hạng bằng 0.
- **Ma trận kích thước lớn (3×3 với hạng 1):** Kiểm tra khả năng trích xuất cơ sở không gian cột từ các cột pivot tương ứng.

```
A_tests = [  
    [[1, 2], [3, 4]],  
    [[1, 2, 3], [0, 4, 5], [0, 0, 6]],  
    [[1, 2], [2, 4]], # rank 1  
    [[0, 0], [0, 0]], # zero matrix  
    [[1, 2, 3], [4, 5, 6]], # 2x3  
    [[1, 2, 3], [2, 4, 6], [3, 6, 9]], # rank 1  
]
```

Hình 9: Các bộ dữ liệu mẫu dùng để kiểm tra hạng và cơ sở không gian con

1.8.2 Kết quả thực thi và Kiểm chứng Kết quả tính toán hạng và các vectơ cơ sở được đối chiếu với các phương pháp chuẩn trong thư viện `numpy`. Toàn bộ các test case đều vượt qua kiểm tra:

```

...   Numpy check: PASSED
      Numpy check: PASSED
      Numpy check: PASSED
      Numpy check: PASSED
      Numpy check: PASSED
      Numpy check: PASSED
      Accuracy: 1.0

```

Hình 10: Kết quả kiểm thử cho thấy thuật toán xác định chính xác cấu trúc ma trận

1.8.3 Nhận xét kết quả:

- **Xác định hạng chính xác:** Chương trình nhận diện đúng số lượng dòng độc lập tuyến tính ngay cả khi các dòng/cột có mối quan hệ phụ thuộc phức tạp (ví dụ: Test case cuối với hạng 1).
- **Tính nhất quán của cơ sở:** Các vectơ cơ sở dòng trích xuất từ dạng bậc thang U và cơ sở cột trích xuất từ ma trận gốc A đảm bảo đúng định nghĩa toán học, tạo thành hệ độc lập tuyến tính tối đại.
- **Xử lý không gian nghiệm (Nullspace):** Thuật toán tìm đúng số lượng vectơ cơ sở cho không gian nghiệm theo định lý về hạng (Rank-Nullity Theorem): $rank(A) + dim(N(A)) = n$.

2 PHÂN RÃ MA TRẬN VÀ KIỂM CHỨNG THỰC NGHIỆM

2.0.1 Chéo hóa ma trận (Diagonalization)

2.1 Cơ sở lý thuyết

Định nghĩa 2.1 (Ma trận chéo hóa được). Ma trận $\mathbf{A} \in \mathbb{R}^{n \times n}$ chéo hóa được nếu tồn tại ma trận $\mathbf{P}$ khả nghịch và ma trận đường chéo $\mathbf{D}$ sao cho:

$$\mathbf{A} = \mathbf{P}\mathbf{D}\mathbf{P}^{-1}, \quad \mathbf{D} = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n) \quad (8)$$

trong đó λ_i là các giá trị riêng và các cột của $\mathbf{P}$ là các vector riêng tương ứng. Ứng dụng quan trọng nhất là tính lũy thừa ma trận bậc cao: $\mathbf{A}^k = \mathbf{P}\mathbf{D}^k\mathbf{P}^{-1}$, giúp giảm độ phức tạp từ $O(n^3 \log k)$ xuống $O(n^2)$.

2.2 Thuật toán lặp QR và Trực giao hóa Gram-Schmidt

Để thực hiện chéo hóa không phụ thuộc thư viện cao cấp, nhóm đã hiện thực hóa thuật toán lặp QR (*QR Iteration*). Động cơ chính của quá trình này là phép trực giao hóa Gram-Schmidt.

2.2.1 Trực giao hóa Gram-Schmidt: Biến đổi tập vectơ thành hệ trực chuẩn thông qua phép chiếu:

```

1 def gram_schmidt(A):
2     for j in range(n):
3         v_j = get_col(v, j)
4         for i in range(j):
5             Q_i = get_col(Q, i)
6             R[i][j] = dot(Q_i, v_j)      # Tính hệ số hình chiếu
7             v_j = vec_sub(v_j, vec_mul(Q_i, R[i][j])) # Khu thành phần
           phụ thuộc
8
9         R[j][j] = norm(v_j)              # Chuẩn hóa vectơ
10        set_col(Q, j, vec_mul(v_j, 1.0 / R[j][j]))
11    return Q, R

```

2.2.2 Vòng lặp QR: Ma trận A_k hội tụ về dạng đường chéo qua các phép biến đổi trực giao liên tiếp:

```

1 def do_diagonalization_primitive(A, num_iters=200):
2     P = eye(n)
3     for _ in range(num_iters):
4         Q, R = gram_schmidt(A_k)
5         A_k = mat_mul(R, Q) # Hội tụ về dạng đường chéo
6         P = mat_mul(P, Q)   # Tích lũy các vectơ riêng
7     return P, get_diag(A_k)

```

3 Phân rã giá trị suy biến (SVD)

3.1 Cơ sở lý thuyết

Phân rã giá trị suy biến (SVD) cho phép phân tích một ma trận bất kỳ $\mathbf{A} \in \mathbb{R}^{m \times n}$ thành tích của ba ma trận: $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$.

- $\mathbf{U} \in \mathbb{R}^{m \times n}$: Ma trận chứa các vectơ suy biến trái (*left singular vectors*).
- $\mathbf{\Sigma} \in \mathbb{R}^{n \times n}$: Ma trận đường chéo chứa các giá trị suy biến $\sigma_i = \sqrt{\lambda_i(\mathbf{A}^T \mathbf{A})}$.
- $\mathbf{V} \in \mathbb{R}^{n \times n}$: Ma trận trực giao chứa các vectơ suy biến phải (*right singular vectors*).

3.2 Hiện thực hóa SVD thủ công

Hàm `do_svd_primitive` kết hợp việc chéo hóa ma trận hiệp phương thức $\mathbf{A}^T \mathbf{A}$ để tìm $\mathbf{V}$ và σ , sau đó suy ra $\mathbf{U}$:

```

1 def do_svd_primitive(A, num_iters=200):
2     # 1. Tìm vecto riêng phải (V) từ AtA
3     AtA = mat_mul(transpose(A), A)
4     V_k, eigenvalues = do_diagonalization_primitive(AtA, num_iters)
5
6     # 2. Tính Singular Values và ma trận U
7     singular_values = [math.sqrt(max(e, 0.0)) for e in eigenvalues]
8     for i in range(n):
9         sigma = singular_values[i]
10        if sigma > 1e-10:
11            u_i = vec_mul(mat_vec_mul(A, V_k[:,i]), 1.0 / sigma)
12            set_col(U, i, u_i)
13    return U, S, transpose(V_k)

```

3.3 Ứng dụng xấp xỉ hạng thấp

Một ứng dụng cực kỳ quan trọng là nén dữ liệu và PCA thông qua xấp xỉ hạng k :

$$\mathbf{A}_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T \quad (8)$$

3.3.1 Kiểm chứng thực nghiệm (Fact-check): Qua đối chiếu với thư viện `numpy.linalg.svd`, thuật toán tự cài đặt đạt sai số tái cấu trúc cực thấp ($\approx 10^{-15}$), khẳng định tính ổn định số học của hệ thống hàm sơ cấp.

4 Phần 3: Giải Hệ Phương Trình và Phân Tích Hiệu Năng

4.1 Lý Thuyết Sai Số và Tính Ổn Định Số

4.1.1 Số Điều Kiện (Condition Number)

Định nghĩa 3.1 (Số điều kiện). *Số điều kiện của ma trận $\mathbf{A}$ đối với chuẩn p là:*

$$\kappa_p(\mathbf{A}) = \|\mathbf{A}\|_p \cdot \|\mathbf{A}^{-1}\|_p. \quad (9)$$

Đối với chuẩn spectral (chuẩn 2): $\kappa_2(\mathbf{A}) = \frac{\sigma_{\max}(\mathbf{A})}{\sigma_{\min}(\mathbf{A})}$.

Định lý 3.1 (Phân tích sai số nghiệm). *Nếu $\mathbf{Ax} = \mathbf{b}$ là nghiệm đúng và $\mathbf{A}\hat{\mathbf{x}} = \mathbf{b} + \delta\mathbf{b}$ là nghiệm tính với nhiễu dữ liệu, thì:*

$$\frac{\|\hat{\mathbf{x}} - \mathbf{x}\|}{\|\mathbf{x}\|} \leq \kappa(\mathbf{A}) \cdot \frac{\|\delta\mathbf{b}\|}{\|\mathbf{b}\|}. \quad (10)$$

Khi $\kappa(\mathbf{A})$ lớn, hệ bị điều kiện kém (ill-conditioned): sai số nhỏ trong dữ liệu đầu vào gây ra sai số lớn trong nghiệm.

4.1.2 So Sánh Các Phương Pháp

Phương pháp	Loại	Điều kiện áp dụng	Chi phí	Ổn định
Gauss (Partial Pivot)	Trực tiếp	Mọi $\mathbf{A}$ khả nghịch	$O(n^3)$	Cao
LU với pivot	Trực tiếp	Mọi $\mathbf{A}$ khả nghịch	$O(n^3)$	Cao
QR (Householder)	Trực tiếp	Mọi $\mathbf{A}$ (kể cả chữ nhật)	$O(n^3)$	Rất cao
Cholesky	Trực tiếp	$\mathbf{A}$ đối xứng xác định dương	$O(n^3/6)$	Rất cao
Gauss–Seidel	Lặp	Chéo trội hàng / SPD	$O(k n^2)$	Trung bình

4.1.3 Phương Pháp Lặp Gauss–Seidel

Phân rã $\mathbf{A} = \mathbf{D} + \mathbf{L} + \mathbf{U}$ (đường chéo $\mathbf{D}$, tam giác dưới thuần $\mathbf{L}$, tam giác trên thuần $\mathbf{U}$). Ta có công thức lặp:

$$\mathbf{x}^{(k+1)} = -(\mathbf{D} + \mathbf{L})^{-1}\mathbf{U}\mathbf{x}^{(k)} + (\mathbf{D} + \mathbf{L})^{-1}\mathbf{b}. \quad (11)$$

Nếu ta viết theo từng thành phần thì công thức lặp sẽ có dạng như sau:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)} \right), \quad i = 1, \dots, n. \quad (12)$$

Điều kiện hội tụ: $\mathbf{A}$ chéo trội chặt hàng, tức $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ với mọi i

4.2 Cài đặt Python

1. Khởi tạo dữ liệu thực nghiệm

Để đánh giá thuật toán một cách khách quan, hai loại ma trận đặc biệt được sử dụng:

- **Ma trận Hilbert:** Được tạo bởi hàm `create_hilbert_matrix(n)` với $H_{ij} = 1/(i+j+1)$. Đây là ma trận có tính chất "ill-conditioned" cực hạn, dùng để thử thách tính ổn định số học của các phương pháp trực tiếp như Gauss và SVD.
- **Ma trận SPD:** Được tạo bởi `create_spd_matrix(n)` thông qua tích $\mathbf{M}\mathbf{M}^T + n\mathbf{I}$. Ma trận này đảm bảo tính chéo trội hàng nghiêm ngặt, là điều kiện cần thiết để kiểm chứng sự hội tụ của phương pháp lặp Gauss–Seidel.

2. Cài đặt các hàm cốt lõi

Các đoạn mã quan trọng thực hiện việc tính toán và đánh giá:

```
1 def mat_vec_mul(A, x):
2     if x is None: return None
3     res = []
4     for row in A:
5         val = 0.0
6         for i, v in enumerate(row):
7             # Xu ly truong hop x chua cac doi tuong bieu thuc (
            Expression)
```

```

8         x_i = float(x[i].mp.get("__const__", 0.0)) if hasattr(x[i]
9         ], 'mp') else float(x[i])
10         val += v * x_i
11         res.append(val)
12     return res
13 def vec_norm(v):
14     return math.sqrt(sum(float(x)**2 for x in v))

```

Giải thích: Hàm `mat_vec_mul` đóng vai trò quan trọng trong việc tính toán vector $\mathbf{b}$ từ nghiệm giả định $\mathbf{x}_{true}$ và tính lại $\mathbf{Ax}_{pred}$ để đo sai số. Việc ép kiểu `float` giúp đồng bộ dữ liệu khi các phương pháp như SVD trả về kết quả dưới dạng đối tượng ký hiệu.

```

1 def benchmark(sizes):
2     results = []
3     for n in sizes:
4         A = create_spd_matrix(n)
5         x_true = [random.random() for _ in range(n)]
6         b = mat_vec_mul(A, x_true)
7
8         for name, method in {"Gauss": solve_gaussian, "SVD": solve_svd
9         , "G-S": solve_gauss_seidel}.items():
10             start = time.time()
11             x_pred = method(A, b)
12             exec_time = time.time() - start
13
14             # Tính sai số tương đối (Relative Error)
15             Ax_p = mat_vec_mul(A, x_pred)
16             rel_error = vec_norm(vec_sub(Ax_p, b)) / (vec_norm(b) + 1e
17             -15)
18
19             results.append({"n": n, "Method": name, "Time": exec_time,
20             "Error": rel_error})
21     return pd.DataFrame(results)

```

Giải thích: Hàm `benchmark` thực hiện quy trình khép kín: Khởi tạo hệ phương trình có nghiệm biết trước $\rightarrow$ Giải bằng 3 phương pháp $\rightarrow$ Đo thời gian thực thi $\rightarrow$ Tính sai số tương đối dựa trên chuẩn Euclid của thặng dư ($\|\mathbf{Ax} - \mathbf{b}\|$).

3. Phân tích tính ổn định và Trực quan hóa

Kết quả từ Benchmark được xử lý qua biểu đồ *log-log*. Việc sử dụng thang đo logarit giúp xác nhận độ phức tạp thời gian thực tế:

- Nếu độ dốc của đường biểu diễn tương đương với đường lý thuyết $O(n^3)$, thuật toán cài đặt đạt hiệu quả tối ưu về cấu trúc vòng lặp.

- Sai số trên ma trận Hilbert thường cao hơn nhiều so với ma trận SPD, phản ánh tác động của số điều kiện (κ) lên độ chính xác của nghiệm.

4.3 Kết quả thực nghiệm

Trong phần này, thực hiện các thử nghiệm thực tế để đánh giá hiệu năng và độ ổn định của ba phương pháp: Khử Gauss (Gaussian Elimination), Phân rã SVD (SVD Decomposition) và Lặp Gauss–Seidel.

1. Thử nghiệm Hiệu năng (Benchmarking)

Thử nghiệm được thực hiện trên các ma trận đối xứng xác định dương (SPD) với kích thước n tăng dần từ 20 đến 100. Thời gian thực thi (tính bằng giây) được ghi nhận trong bảng dưới đây:

Kích thước (n)	Gauss (s)	Gauss–Seidel (s)	SVD (s)
20	0.004709	0.003142	0.622341
50	0.034294	0.052663	8.872906
80	0.128175	0.246207	36.850499
100	0.137167	0.438159	62.925034

Nhận xét: Phương pháp Khử Gauss cho tốc độ thực thi nhanh nhất đối với các hệ phương trình trực tiếp. Phương pháp SVD có chi phí tính toán cao nhất do tính chất phức tạp của thuật toán phân rã ma trận trong môi trường Python thuần.

2. Đánh giá Sai số

Sai số tương đối của các phương pháp được tính toán dựa trên chuẩn thặng dư $\|\mathbf{Ax} - \mathbf{b}\|$. Kết quả cho thấy phương pháp Gauss duy trì độ chính xác cực cao ($\approx 10^{-16}$), tiệm cận giới hạn sai số làm tròn của máy tính.

Kích thước (n)	Gauss	Gauss–Seidel	SVD
20	1.38×10^{-16}	1.32×10^{-11}	4.64×10^{-4}
100	2.85×10^{-16}	2.26×10^{-11}	2.26×10^{-4}

3. Kiểm chứng độ ổn định (Stability Analysis)

Để kiểm tra khả năng chịu lỗi, thuật toán được thử nghiệm với ma trận Hilbert ($n = 5$), vốn có số điều kiện rất lớn khiến hệ phương trình trở nên khó giải (ill-conditioned).

Phương pháp	Sai số (Ma trận Hilbert)	Sai số (Ma trận SPD)
Gauss	1.68×10^{-14}	1.11×10^{-16}
SVD	1.125	1.17×10^{-3}

Kết luận: Kết quả thực nghiệm xác nhận rằng phương pháp Khử Gauss với kỹ thuật chọn phần tử chốt (Partial Pivoting) đạt được sự cân bằng tối ưu giữa tốc độ thực thi và độ ổn định số học. Trong khi đó, biểu đồ Log-Log (Hình ...) cho thấy độ dốc của các đường thực nghiệm bám sát đường lý thuyết $O(n^3)$, chứng minh tính đúng đắn trong việc cài đặt thuật toán.

5 KẾT LUẬN

5.1 Tóm tắt kết quả đạt được

- **Xây dựng thành công bộ công cụ giải toán:** Đã cài đặt hoàn chỉnh các thuật toán nền tảng bao gồm khử Gauss (Partial Pivoting), phân rã SVD và phương pháp lặp Gauss–Seidel bằng ngôn ngữ Python thuần.
- **Tối ưu hóa độ chính xác:** Thuật toán khử Gauss đạt độ chính xác cực cao với sai số tiệm cận mức 10^{-16} trên các ma trận SPD.
- **Phân tích hiệu năng chi tiết:** Thông qua thực nghiệm Benchmarking và biểu đồ Log–Log, đã chứng minh được sự tương quan chặt chẽ giữa thời gian thực thi thực tế và độ phức tạp lý thuyết $O(n^3)$.
- **Trực quan hóa báo cáo:** Hoàn thiện việc chuyển đổi dữ liệu phân tích từ môi trường lập trình (Jupyter Notebook) sang định dạng $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ chuyên nghiệp, bao gồm các bảng biểu và công thức toán học chuẩn xác.

5.2 Khó khăn gặp phải

- **Độ ổn định số học:** Khi xử lý các ma trận có điều kiện kém (ill-conditioned) như ma trận Hilbert, sai số làm tròn máy tính bị khuếch đại mạnh, gây khó khăn cho việc tìm nghiệm chính xác bằng phương pháp SVD tự cài đặt.
- **Hiệu năng ngôn ngữ:** Việc sử dụng Python thuần (không dùng thư viện tối ưu như NumPy) khiến thời gian thực thi của các thuật toán phức tạp như SVD tăng nhanh khi kích thước ma trận lớn ($n > 100$).
- **Quản lý mã nguồn:** Trong quá trình làm việc nhóm, việc xảy ra xung đột (conflict) và tình trạng rẽ nhánh (divergent branches) trên Git gây mất thời gian trong việc gộp mã nguồn và đồng bộ hóa trình soạn thảo hệ thống.

5.3 Bài học rút ra

- **Kỹ năng lập trình toán học:** Hiểu rõ tầm quan trọng của việc chọn phần tử chốt (pivoting) trong phép khử Gauss để giảm thiểu sai số máy tính và đảm bảo tính ổn định của thuật toán.
- **Kỹ năng làm việc nhóm:** Thành thạo quy trình xử lý lỗi Git (Pull, Merge, Rebase) và cấu hình môi trường làm việc đồng bộ trên các hệ điều hành khác nhau.
- **Kỹ năng trình bày khoa học:** Học được cách tích hợp hiệu quả giữa mã nguồn, kết quả phân tích thực nghiệm và trình bày báo cáo bằng $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ /Markdown để tạo ra sản phẩm chuyên môn có tính thẩm mỹ và độ tin cậy cao.

- **Xây dựng thương hiệu cá nhân:** Biết cách sử dụng GitHub Profile README để trình bày năng lực kỹ thuật (Tech Stack) và kết quả đóng góp dự án một cách trực quan.

III TÀI LIỆU THAM KHẢO

- [1] Gilbert Strang. *Introduction to Linear Algebra*, 6th ed. Wellesley-Cambridge Press, 2023.
- [2] Lloyd N. Trefethen & David Bau III. *Numerical Linear Algebra*. SIAM, 1997.
- [3] Gene H. Golub & Charles F. Van Loan. *Matrix Computations*, 4th ed. Johns Hopkins University Press, 2013.
- [4] Cleve Moler. *Numerical Computing with MATLAB*. SIAM, 2004.
<https://www.mathworks.com/moler/chapters.html>
- [5] Manim Community Developers. *Manim Framework*, v0.18. 2024.
<https://docs.manim.community>
- [6] Jake VanderPlas. *Python Data Science Handbook*. O'Reilly, 2016.
<https://jakevdp.github.io/PythonDataScienceHandbook/>
- [7] 3Blue1Brown. *Essence of Linear Algebra*. YouTube, 2016.
https://www.youtube.com/playlist?list=PLZHQOb0WTQDPD3MizzM2xVFitgF8hE_ab